Calidad de software

Estudiantes

Karen Jimenez Espinoza

208090539

Kendall Reyes Solano

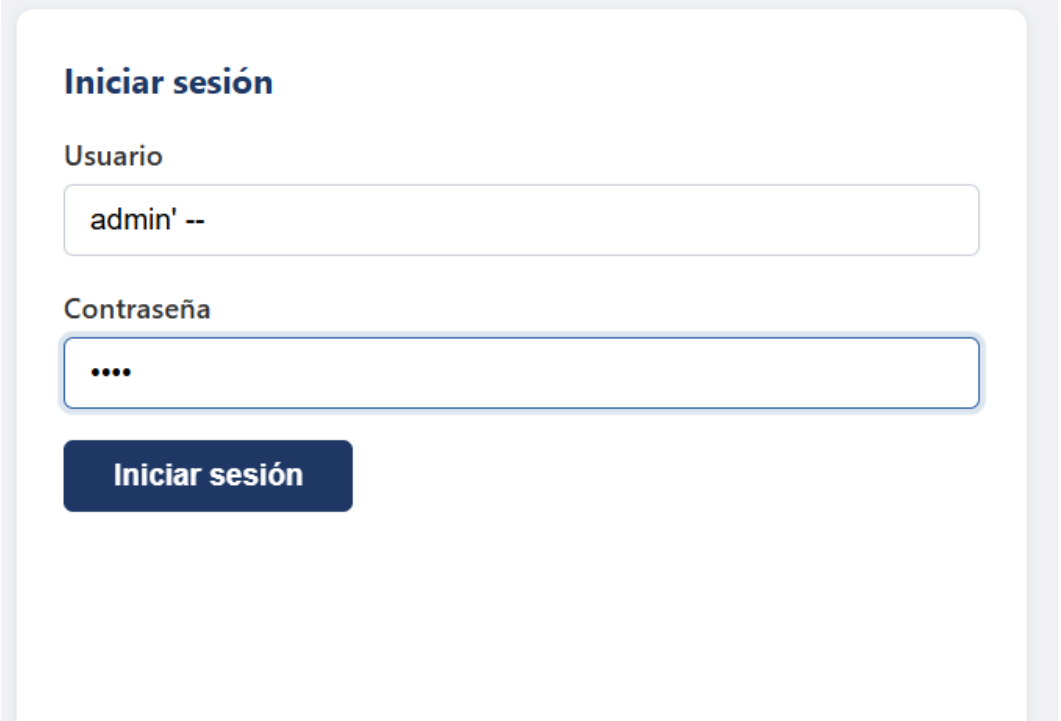
208500837

23/03/2026

Karen Jimenez
Kendall Reyes

Documentación de los ataques

Ataque 1: Bypass de login sin contraseña (V-01):



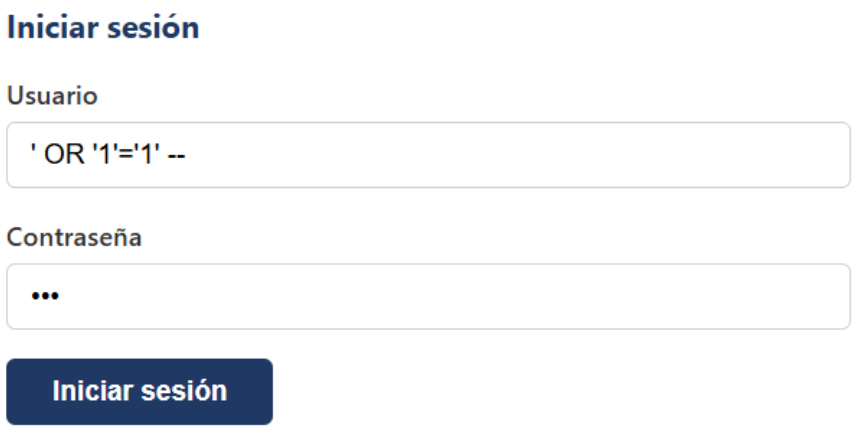
Resultado

Explicación: Se realiza el ataque de sql-injection en donde se ingresa con con el método de (admin' - -) que el doble guión comenta el resto: la condición de contraseña se ignora.

Ataque 2: Login sin conocer ningún usuario (V-01)

Karen Jimenez
Kendall Reyes

Ataque



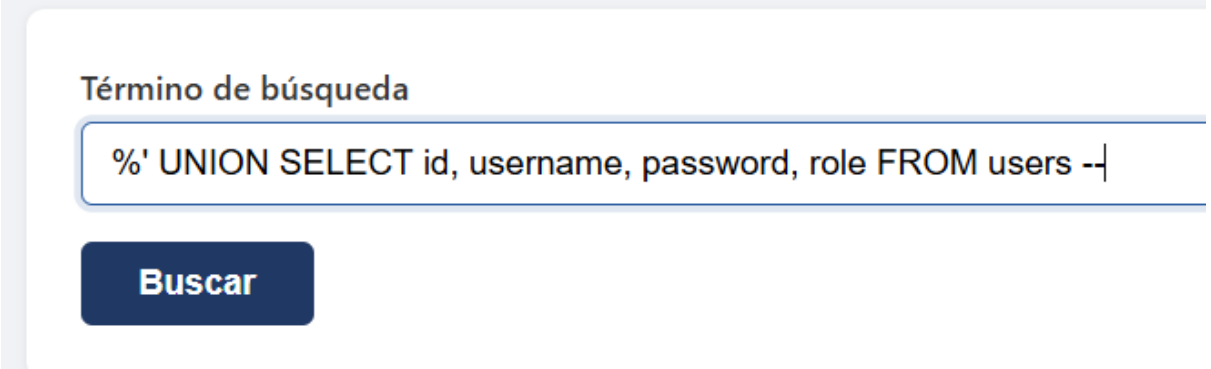
Resultado (Permitió entrar con el usuario admin)

Este ataque lo que permite es que la condición WHERE siempre sea verdadera y retorna el primer usuario de la tabla.

Karen Jimenez
Kendall Reyes

Ataque 3: Extraer usuarios con UNION (V-02)

Ataque



Resultado

Resultados (11 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
1	admin	Admin123	admin
2	Flask Web Patterns	A. Miller	development
2	analyst	Analyst123	user
3	Practical SQL	A. DeBarros	database
3	student	Student123	user
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

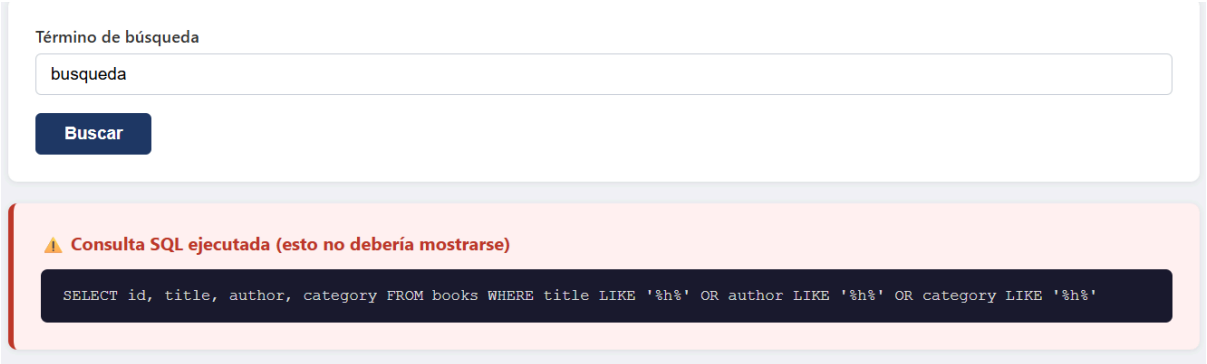
Los resultados muestran los registros de la tabla users con sus contraseñas en texto plano, en lugar de libros.

Correcciones: Se construye el patrón LIKE en Python con f-string normal, # y se pasa como parámetro con ? — SQLite lo trata como dato, # nunca como código SQL.

Ataque 4: Verificar consulta expuesta (V-06)

Karen Jimenez
Kendall Reyes

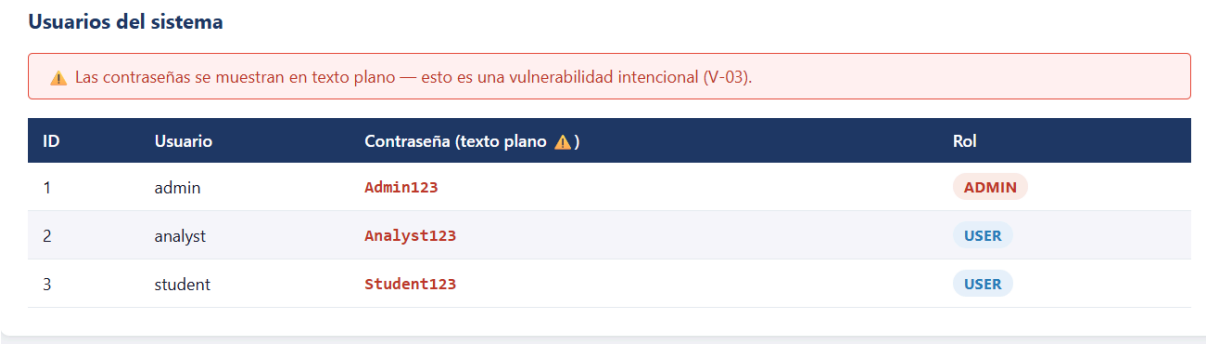
Vulnerabilidad



En la búsqueda se ingresa el término búsqueda y se observa debajo del formulario una consulta SQL. Esto nunca debería ser visible para el usuario.

Ataque 5: Contraseñas en texto plano (V-03)

Vulnerabilidad



Se ingresa como admin y en el panel de Admin se pueden ver completamente las contraseñas en texto plano. Lo cual es una vulnerabilidad.

Documentación de soluciones

V-01	**SQLi Login**	Consulta construida con f-string concatenando el input del usuario.	Reemplazar con consulta parametrizada usando ? como placeholder.

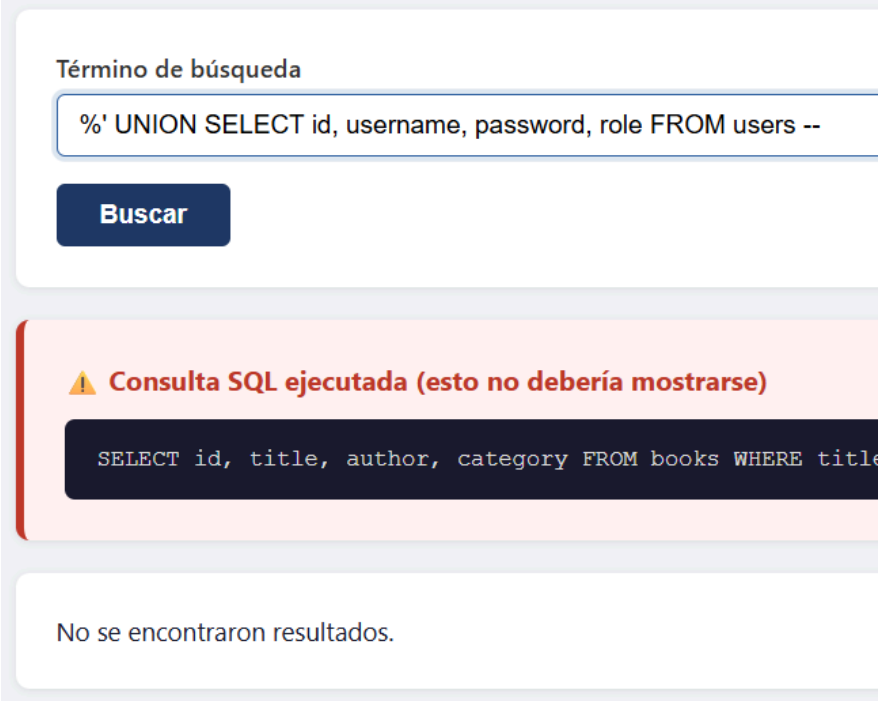
Resolución: Se reemplazó la consulta por la que el servidor accede a la base de datos por una consulta parametrizada.

```
query = "SELECT id, username, role FROM users WHERE username = ? AND
password = ?"
```

Karen Jimenez
Kendall Reyes

Karen Jimenez
Kendall Reyes

V-02	**SQLi Búsqueda**	Mismo problema con LIKE: el término se inserta directamente.	Usar consulta parametrizada con ? y pasar el valor con %...% en Python.



V-04	**SECRET_KEY fija**	La clave secreta está hardcodeada en el código fuente.	Cargarla desde una variable de entorno con os.environ.get().

Problema:

```
# V-04: SECRET_KEY hardcodeada en el código fuente.
# En una aplicación real debe cargarse desde una variable de
# entorno y nunca commitearse al repositorio.
# ----------------------------------------------------------------
app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

Solución:

- Se corre este comando en bash: export SECRET_KEY="dev-secret-key-insegura-1234" python app.py

-import os

-Cambiamos el código:

Karen Jimenez
Kendall Reyes

secret_key = os.environ.get("SECRET_KEY")
if not secret_key:
    raise RuntimeError("La variable de entorno SECRET_KEY no está definida.")

app.config["SECRET_KEY"] = secret_key

V-05	**debug=True**	El debugger de Flask está activo y permite ejecutar código en el servidor.	Cambiar a debug=False o leer el valor desde una variable de entorno.

Código con error:

```
# ----------------------------------------------------------------
# V-05: debug=True activo.
# En modo debug, Flask activa un debugger interactivo en el
# navegador cuando ocurre un error. Cualquier visitante puede
# ejecutar código Python arbitrario en el servidor.
# Nunca debe usarse debug=True en producción.
# ----------------------------------------------------------------
if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Cambio de código/solucion:

if __name__ == "__main__":
    init_db()
    debug_mode = os.environ.get("FLASK_DEBUG", "false").lower() == "true"
    app.run(host="0.0.0.0", port=5000, debug=debug_mode)

V-06	**Query expuesta**	El template muestra la consulta SQL ejecutada en pantalla.	Eliminar el bloque que renderiza raw_query en search.html.

Código con error:

```
    # V-06: La consulta SQL cruda se pasa al template y se
    # muestra en pantalla — expone la estructura interna de la BD.
    return render_template("search.html", books=books,
raw_query=raw_query)
```

Código con solución:
Se elimina raw_query del template.Ya no se expone la estructura interna de la BD en pantalla.

return render_template("search.html", books=books)

Karen Jimenez
Kendall Reyes

V-07	**Menú Admin**	El enlace Admin aparece para todos los roles en la navegación.	En layout.html, mostrar el enlace Admin solo si session.get('role') == 'admin'.

Código:
En layout.html modificamos:

Antes= visible para todos
<a href="{{ url_for('admin') }}">Admin</a>

Después:

```
{% if session.get('role') == 'admin' %}
  <a href="{{ url_for('admin') }}">Admin</a>
{% endif %}
```

Conclusión:

En este minilab se aplicaron diversos métodos de seguridad sobre código con distintas vulnerabilidades que permitían hacer SQL Injections.
Se conocieron algunas formas de escribir consultas SQL dirigidas a la base de datos de forma que permitan ataques de SQL injection.